
Dossier de déploiement et mise en production

Jalon 6 – Projet Fil Rouge CDA

Étudiant FORTUNATO Axel

Formation IPSSI — CDA Bachelor Fullstack DevOps

Promotion 2025–2026

Date Juin 2026

Table des matières

1. Objectif du jalon 6	3
2. Périmètre livré et stack technique	4
2.1 Fonctionnalités (CDCF)	4
2.2 Conformité réglementaire (RGPD)	4
2.3 Stack et écarts assumés	4
3. Configuration et environnements	6
3.1 Variables d'environnement	6
3.2 Environnements prod, dev et test	7
3.3 phpMyAdmin	8
4. Installation et procédure de déploiement	9
4.1 Prérequis et démarrage standard	9
4.2 Comptes de démonstration	10
4.3 Déploiement sur un serveur	10
4.4 Opérations courantes	10
5. Stratégie de mise en production et DevOps	12
5.1 Intégration continue vs déploiement continu	12
5.2 Stratégies de déploiement	12
5.3 Retour d'expérience DevOps	13
6. Architecture déployée	14
6.1 Schéma Docker Compose	14
6.2 Rôle des composants	14
7. Sécurité et conformité	15
7.1 Mesures OWASP	15
7.2 RGPD	15

- 8. Tests et intégration continue 17
 - 8.1 Périmètre des tests 17
 - 8.2 Exécution locale et pipeline CI 17
- 9. Guide utilisateur et scénario de démonstration 19
 - 9.1 Parcours parent 19
 - 9.2 Parcours administrateur 19
- 10. Conclusion du jalon 6 20

1. Objectif du jalon 6

Ce document clôt le **jalón 6** du projet fil rouge CDA : **déploiement et livrable final** de l'application **Storybook Kids**. Il correspond au chapitre XI du CDC technique et synthétise tout ce qu'un jury ou un développeur tiers doit savoir pour exécuter, déployer et démontrer l'application.

Il couvre :

- le **périmètre fonctionnel** livré en version finale,
- la **configuration** (variables d'environnement, environnements prod / dev / test),
- la **procédure de déploiement** reproductible (Docker Compose),
- l'**architecture déployée**, la **sécurité**, les **tests** et la **CI**,
- un **guide utilisateur** pour la démonstration orale,
- un **retour d'expérience DevOps** et une **conclusion** avec perspectives.

Dépôt Git : https://github.com/FortAxel/ipssi_project

2. Périmètre livré et stack technique

2.1 Fonctionnalités (CDCF)

ID	Domaine	Détail
F1	Auth & profil	Inscription, JWT, modification e-mail / mot de passe, suppression de compte
F2	Catalogue	Liste, détail, recherche, filtres par catégorie
F3	Lecture	Pages, navigation, progression X/Y, reprise à la dernière page
F4	Favoris	Toggle, page dédiée, historique (Profil)
F5	Admin	CRUD histoires / pages, upload images, gestion utilisateurs
F6	TTS	Edge TTS (POST /api/tts/synthesize) + secours Web Speech API

Contenu de démonstration : **5 histoires** en français, chargées via fixtures Doctrine au premier démarrage si la base est vide.

2.2 Conformité réglementaire (RGPD)

Exigence	Implémentation
Politique de confidentialité	Route /privacy
Rectification des données	PATCH /api/me
Suppression de compte	DELETE /api/me
Sécurisation des accès	Mots de passe hachés, API stateless JWT

2.3 Stack et écarts assumés

Couche	Technologie
API	Symfony 7.2, PHP 8.4, JWT (Lexik)
Données	MySQL 8, Doctrine ORM, migrations versionnées
Front	React 19, TypeScript, Vite
Infra	Docker Compose — nginx, PHP-FPM, MySQL, phpMyAdmin, app-init
Qualité	PHP-CS-Fixer, ESLint, PHPUnit, Vitest, GitHub Actions

Écarts documentés par rapport au CDCF ou à la conception initiale :

Sujet	Choix retenu
TTS (Google / Polly cités au CDCF)	Edge TTS — API externe gratuite
Tests front (Jest + RTL)	Vitest
CSRF Symfony (formulaires Twig)	API stateless JWT ; protection Bearer + CORS
Modération utilisateurs in-app	Absente ; phpMyAdmin pour la BDD
Déploiement continu (CD)	Non implémenté ; procédure manuelle documentée §4
Hébergement cloud public	Non requis ; Docker documenté pour serveur ou poste local

3. Configuration et environnements

3.1 Variables d'environnement

Fichier unique à la racine du dépôt : `.env` (copier depuis `.env.example`, monté dans le conteneur PHP).

Variable	Rôle	Défaut / remarque
APP_ENV	Environnement Symfony	prod ou dev
APP_DEBUG	Mode debug	0 en prod
APP_PORT	Port HTTP application	8080
PHPMYADMIN_PORT	Port phpMyAdmin	8081
DEFAULT_URI	URL publique API	alignée sur APP_PORT
APP_SECRET	Secret Symfony	≥ 32 caractères aléatoires en prod
MYSQL_*	Identifiants MySQL	voir <code>.env.example</code>
DATABASE_URL	Doctrine	mot de passe = MYSQL_PASSWORD, hôte mysql
JWT_PASSPHRASE	Clé privée JWT	ne pas changer après 1er boot sans régénérer les clés
CORS_ALLOW_ORIGIN	Origines autorisées	regex localhost / 127.0.0.1
TTS_*	Synthèse vocale Edge	TTS_ENABLED=1 par défaut

Points clés :

- le mot de passe dans `DATABASE_URL` doit correspondre à `MYSQL_PASSWORD` ;
- `DEFAULT_URI` doit refléter l'URL réelle (ex. `http://localhost:8080` ou URL publique) ;
- la liste complète avec commentaires figure dans `.env.example` à la racine du dépôt.

3.2 Environnements prod, dev et test

	Docker prod	Docker dev	Tests CI
Usage	Livraison, démo jury, serveur	Debug Symfony	GitHub Actions / PHPUnit local
Config	APP_ENV=prod APP_DEBUG=0	APP_ENV=dev APP_DEBUG=1	APP_ENV=test
Frontend Build	nginx :8080	nginx :8080 ou npm run dev → :5173	Vitest sur l'hôte
Backend	Composer --no-dev	Composer + dev dependencies	SQLite (backend/.env.test)
BDD	MySQL (volume mysql_data)	MySQL	SQLite
phpMyAdmin	Oui	Oui	—

Mode prod — application complète sur :8080, sans Node.js sur le serveur :

```
APP_ENV=prod
APP_DEBUG=0
```

```
docker compose up -d --build
```

Mode dev — hot-reload React (Node.js 20+ requis sur l'hôte) :

```
APP_ENV=dev
APP_DEBUG=1
```



```
docker compose up -d --build  
cd frontend && npm install && npm run dev
```

→ API sur :8080, interface React sur http ://127.0.0.1 :5173.

3.3 phpMyAdmin

Inclus dans la stack Docker (port 8081). L'application n'a pas d'écran de modération utilisateurs : phpMyAdmin permet de consulter ou corriger les données en base (comptes, contenus) en développement comme en production.

4. Installation et procédure de déploiement

Le déploiement est **manuel** et **reproductible**. Aucun dump SQL séparé n'est nécessaire : les migrations Doctrine et les fixtures initialisent la base au premier boot.

4.1 Prérequis et démarrage standard

Outil	Version minimale
Git	2.x
Docker	24+
Docker Compose	v2 (docker compose)

```
git clone https://github.com/FortAxel/ipssi_project.git
cd ipssi_project

cp .env.example .env
# Éditer .env : APP_SECRET, JWT_PASSPHRASE, mots de passe MySQL en production

docker compose up -d --build
```

Au premier démarrage, le conteneur app-init installe les dépendances PHP, applique les migrations et charge les fixtures si la base est vide.

Vérification :

```
curl -s http://127.0.0.1:8080/api/health
# → {"status":"ok","service":"storybook-kids-api"}
```

Service	URL par défaut
Application (SPA + API)	http ://127.0.0.1 :8080
phpMyAdmin	http ://127.0.0.1 :8081

Service	URL par défaut
MySQL (client externe)	127.0.0.1:3307

4.2 Comptes de démonstration

Rôle	E-mail	Mot de passe
Parent	parent@demo.local	parent123
Admin	admin@demo.local	admin123

4.3 Déploiement sur un serveur

1. Cloner le dépôt sur le serveur (branche main ou tag release v1.0.0).
2. Copier et personnaliser `.env` : secrets forts, `DEFAULT_URI` = URL publique.
3. Ouvrir le port `APP_PORT` (défaut 8080) ; restreindre `PHPMYADMIN_PORT` au réseau admin si exposé.
4. Lancer `docker compose up -d --build`.
5. Vérifier `GET /api/health` sur l'URL publique.
6. Placer un reverse proxy HTTPS (Nginx, Caddy, Traefik) devant `APP_PORT` (recommandé).

Mise à jour d'une version existante :

```
git pull
docker compose up -d --build
```

4.4 Opérations courantes

```
# Rebuild frontend seul (après modification React en prod)
docker compose up -d --build nginx

# Rebuild complet
docker compose up -d --build
```

```
# Réinitialiser la base (conteneurs + volume)
```

```
docker compose down -v
```

```
docker compose up -d --build
```

```
# Arrêter la stack
```

```
docker compose down
```

```
# Logs
```

```
docker compose logs -f
```

```
docker compose logs -f php
```

JWT — si login ou inscription renvoie une erreur 500 après changement de JWT_PASSPHRASE :

```
docker compose exec php php bin/console lexik:jwt:generate-keypair --overwrite --no-interaction
```

5. Stratégie de mise en production et DevOps

5.1 Intégration continue vs déploiement continu

Volet	Mise en œuvre
CI	GitHub Actions (<code>.github/workflows/ci.yml</code>) — à chaque push/PR sur <code>main</code> ou <code>develop</code> : lint PHP, PHPUnit, lint front, Vitest, build React
CD	Non automatisé vers un serveur distant ; déploiement manuel <code>docker compose up -d --build</code> sur la cible

Évolution possible (non réalisée) : pipeline CI construisant une image Docker, push sur un registre (Docker Hub, GHCR), puis script serveur `docker pull && docker compose up -d`.

5.2 Stratégies de déploiement

Stratégie retenue pour ce projet : **recreate** en fenêtre de maintenance — `docker compose down`, pull du code, `docker compose up -d --build`, vérification healthcheck. L'interruption de service est acceptable pour une démo ou une mise à jour ponctuelle.

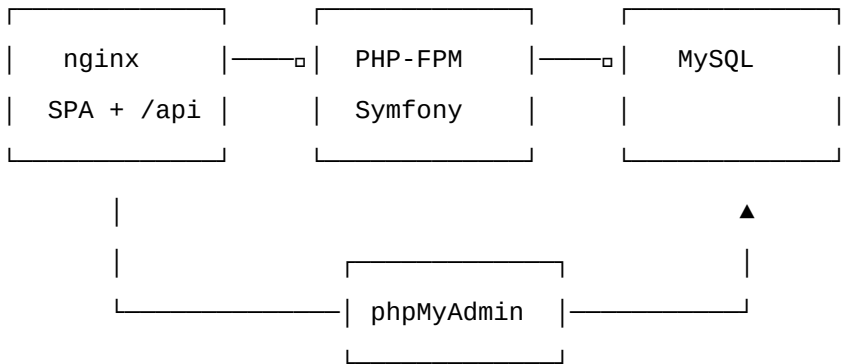
Stratégie	Principe	Applicabilité au projet
Recreate (<i>actuel</i>)	Arrêt → rebuild → redémarrage	Simple, documenté, suffisant ici
Blue/Green	Deux environnements identiques ; bascule du routage après validation	Reverse proxy / load balancer, double stack
Rolling update	Mise à jour progressive conteneur par conteneur	Orchestrateur multi-replicas (Kubernetes, Swarm)

5.3 Retour d'expérience DevOps

Difficulté rencontrée	Résolution
Dépendances Symfony en require-dev	Packages runtime (Doctrine, JWT, Security...) déplacés en require pour composer install --no-dev en prod
Clés JWT vs JWT_PASSPHRASE	Script docker/init/bootstrap.sh régénère la paire si passphrase incompatible
Fixtures en production	FixturesBundle activé ; seed au premier boot si BDD vide
Stack dev vs prod	Un seul docker-compose.yml ; bascule via APP_ENV dans .env
Frontend sans Node sur le serveur	Build React intégré à l'image nginx — application complète sur :8080
Réinitialisation BDD	docker compose down -v puis rebuild

6. Architecture déployée

6.1 Schéma Docker Compose



6.2 Rôle des composants

Composant	Rôle
nginx	Fichiers statiques React (build Vite) + proxy /api et /images vers Symfony
php	API REST Symfony 7.2 (JWT, Doctrine, TTS)
app-init	Conteneur one-shot : Composer, migrations, fixtures, clés JWT si absentes
mysql	Persistence des données — volume Docker mysql_data
phpmyadmin	Administration BDD (données, modération sans écran dédié in-app)

Couches logiques : React (présentation) → API Symfony (métier) → Doctrine (accès données) → MySQL.

Fichiers d'infrastructure : docker-compose.yml, docker/nginx/, docker/php/, docker/init/bootstrap.sh.

7. Sécurité et conformité

7.1 Mesures OWASP

Risque	Mesure
Injection SQL	Doctrine ORM, requêtes paramétrées
XSS	Échappement React; pas de dangerouslySetInnerHTML sur le contenu des histoires
Authentification	JWT (Lexik), mots de passe hachés (algorithme Symfony)
Autorisation	ROLE_USER / ROLE_ADMIN; routes /api/admin/* réservées admin
Fuite de données	Progression et favoris liés au user_id du token; tests d'isolation API
Secrets	.env et clés JWT (config/jwt/*.pem) hors dépôt (gitignore)
CORS	Nelmio — origines configurables via CORS_ALLOW_ORIGIN
Upload fichiers	Types MIME autorisés, taille max 5 Mo, noms aléatoires
TTS	Pas de clé API tierce payante; texte non stocké hors flux de lecture

CSRF : l'API est consommée en SPA + Bearer token; les mécanismes CSRF Symfony (formulaires Twig) ne s'appliquent pas aux routes JSON stateless.

7.2 RGPD

Droit / obligation	Implémentation
Transparence	Page /privacy
Accès / rectification	Profil utilisateur; PATCH /api/me

Droit / obligation	Implémentation
Effacement	DELETE /api/me
Minimisation	Pas de compte enfant ; données limitées au compte parent

Pas de scan automatique OWASP (ZAP) en CI — revue manuelle documentée ci-dessus.

8. Tests et intégration continue

8.1 Périmètre des tests

Suite	Outil	Couverture
Backend unitaire	PHPUnit	Entité ReadingProgress
Backend fonctionnel	PHPUnit + WebTestCase	Health, auth, compte, favoris, progression lecture
Frontend	Vitest	Libellés i18n français, composants clés
Lint	PHP-CS-Fixer, ESLint	Conventions (docs/code-standard.md)
Build	Vite	Compilation production du SPA en CI

8.2 Exécution locale et pipeline CI

Local (identique à la CI) :

```
cd backend
cp .env.test .env
composer install --no-interaction --prefer-dist
composer lint:php
JWT_PASSPHRASE=ci_test_passphrase php bin/console lexik:jwt:generate-keypair --overwrite --no-in
APP_ENV=test composer test

cd frontend
npm ci && npm run lint && npm test && npm run build
```

Note : docker compose exec php composer test charge le .env racine (prod/dev), pas l'environnement test SQLite. La CI et les commandes ci-dessus sont la référence.

Pipeline CI (.github/workflows/ci.yml) :

- **Déclencheur** : push et pull request sur main, develop, dev/beta_usable.
- **Backend** : PHP 8.4, composer install, PHP-CS-Fixer, génération clés JWT test, PHPUnit.

- **Frontend** : Node 20, npm ci, ESLint, Vitest, npm run build.
-

9. Guide utilisateur et scénario de démonstration

Le jury joue le rôle d'un utilisateur durant la démo. Durée indicative : ~10 min de parcours applicatif (+ présentation architecture en soutenance).

9.1 Parcours parent

Compte : parent@demo.local / parent123

1. **Connexion** → accès au catalogue (5 histoires, anneau de progression si déjà lu).
2. **Recherche / filtres** — barre de recherche et filtres par catégorie.
3. **Lecture** — ouvrir une histoire, naviguer page par page, bouton **Écouter** (TTS si TTS_ENABLED=1).
4. **Reprise** — fermer et rouvrir : reprise à la dernière page lue.
5. **Favoris** — icône cœur ; consulter la page **Favoris**.
6. **Profil** — modifier e-mail ou mot de passe ; consulter **Historique** ; supprimer le compte si besoin.
7. **Confidentialité** — lien vers /privacy.

9.2 Parcours administrateur

Compte : admin@demo.local / admin123

1. **Administration** → créer une histoire (titre, description, couverture, statut PUBLISHED).
 2. **Gérer les pages** → ajouter texte et illustration par page.
 3. **Gestion utilisateurs** → consulter les comptes.
 4. Vérifier la nouvelle histoire dans le catalogue parent.
-

10. Conclusion du jalon 6

À l'issue de ce jalon, l'application **Storybook Kids** est **100 % fonctionnelle et déployable** : un jury peut cloner le dépôt, configurer `.env`, lancer `docker compose up -d --build` et démontrer les parcours parent et administrateur sans assistance.

Apports principaux :

- stack Docker unifiée (SPA buildée + API + MySQL + phpMyAdmin) ;
- procédure de mise en production manuelle documentée ;
- durcissement RGPD, tests API élargis, pipeline CI ;
- guide utilisateur pour la maintenance.

Difficultés surmontées : alignement Composer prod/dev, bootstrap JWT et fixtures, containerisation avec frontend buildé sans Node sur le serveur.

Perspectives d'évolution (hors périmètre livré) :

- export des données personnelles (portabilité RGPD) ;
- écran de modération utilisateurs in-app ;
- déploiement continu automatisé (registry Docker) ;
- couverture de tests élargie (E2E Playwright, scan OWASP ZAP) ;
- hébergement cloud avec HTTPS managé et URL de démo permanente.

Le rapport final PDF (chapitres III à XI) consolidera ce dossier avec les livrables des jalons précédents, mis à jour pour l'état final du projet.